

ATmega32A Module

UART (Serial Communication)

Ali Sahafi

1 Theory

The UART (*Universal Asynchronous Receiver/Transmitter*) sends bytes over a single wire per direction: **TXD** (PD1) transmits, **RXD** (PD0) receives. “Asynchronous” means there is no clock wire — both sides must agree on the **baud rate** (bits per second) in advance. Each byte travels as a *frame*: 1 start bit, 8 data bits, 1 stop bit (“8N1”), so at 9600 baud you get at most 960 bytes/s.

The hardware generates the baud rate by dividing the CPU clock. This driver always enables **double speed mode** (U2X), which halves the divider and reduces rounding error:

$$\text{UBRR} = \frac{F_{\text{CPU}}}{8 \cdot \text{baud}} - 1$$

This is why a correct `F_CPU` is critical: if it doesn’t match the real clock, the baud rate is wrong and the terminal shows garbage.

2 Registers

Register	Role	Bits used by the driver
UBRRH/UBRRL	Baud rate divider	12-bit UBRR value
UCSRA	Status	U2X, UDRE (TX ready), RXC (byte received)
UCSRB	Enable	RXEN, TXEN, RXCIE (RX interrupt)
UCSRC	Frame format	URSEL, UCSZ1:0 = 8 data bits
UDR	Data	write to send, read to receive

3 API reference

Method	Description
<code>UART.init(baud, interrupt)</code>	Configure baud rate, optional RX interrupt
<code>UART.transmit(byte)</code>	Send raw byte
<code>UART.print(value)</code>	Send string, char, int, uint or float (overloaded)
<code>UART.print(float, decimals)</code>	Float with chosen decimal places (default 2)
<code>UART.receive()</code>	Blocking receive — waits for incoming byte
<code>UART.attachRxInterrupt(callback)</code>	Set RX callback: <code>void cb(uint8_t byte)</code>

Receiving can be **blocking** (`receive()` halts until a byte arrives) or **interrupt-driven** (pass `true` to `init` and attach a callback — bytes then arrive “by themselves” while your main loop does other work; remember `sei()`).

4 Example: serial echo + uptime printer

Wiring (a USB–serial adapter, e.g. CP2102 or FT232):

- PD0 (RXD) $\leftarrow$ adapter TX
- PD1 (TXD) $\rightarrow$ adapter RX
- GND $\leftrightarrow$ GND (common ground is mandatory!)

Open a serial terminal at **9600 baud, 8N1**.

Build & flash. make uart

```
/*
 * UART Example -- serial echo with RX interrupt + print() showcase
 *
 * Open a serial terminal at 9600 baud (8N1). Every character you type is
 * echoed back by the RX interrupt, and a counter is printed once per second.
 *
 * Wiring:
 * - PD0 (RXD) -> TX of your USB-serial adapter
 * - PD1 (TXD) -> RX of your USB-serial adapter
 * - GND <-> GND
 *
 * Build & flash: make uart
 */

#define F_CPU 8000000UL
#include "uart.hpp"

void onReceive(uint8_t byte) {
    UART.print("Echo: ");
    UART.transmit(byte);
    UART.print("\r\n");
}

int main() {
    UART.init(9600, true); // true = enable RX interrupt
    UART.attachRxInterrupt(onReceive);
    sei();

    UART.print("UART ready! Type something...\r\n");

    uint16_t seconds = 0;
    while (true) {
        _delay_ms(1000);
        seconds++;

        // print() is overloaded for strings, chars, ints and floats:
        UART.print("Uptime: ");
        UART.print(seconds);
        UART.print(" s (");
        UART.print(seconds / 60.0, 2);
        UART.print(" min)\r\n");
    }

    return 0;
}
```

5 Exercises

1. Turn the echo into a *command interpreter*: '1' switches an LED on, '0' switches it off, anything else prints "?".
2. Compute UBRR for 9600 baud at $F_{\text{CPU}} = 1\text{ MHz}$ (the factory default clock!) with U2X enabled. How large is the rounding error, and what do you expect to see in the terminal?
3. Combine with Module 5: print the ADC value of the potentiometer once per second, and use a serial-plotter tool to graph it live.